Niharika Gupta

192425013

ITA0504 – Computer Vision

Short Answer programs

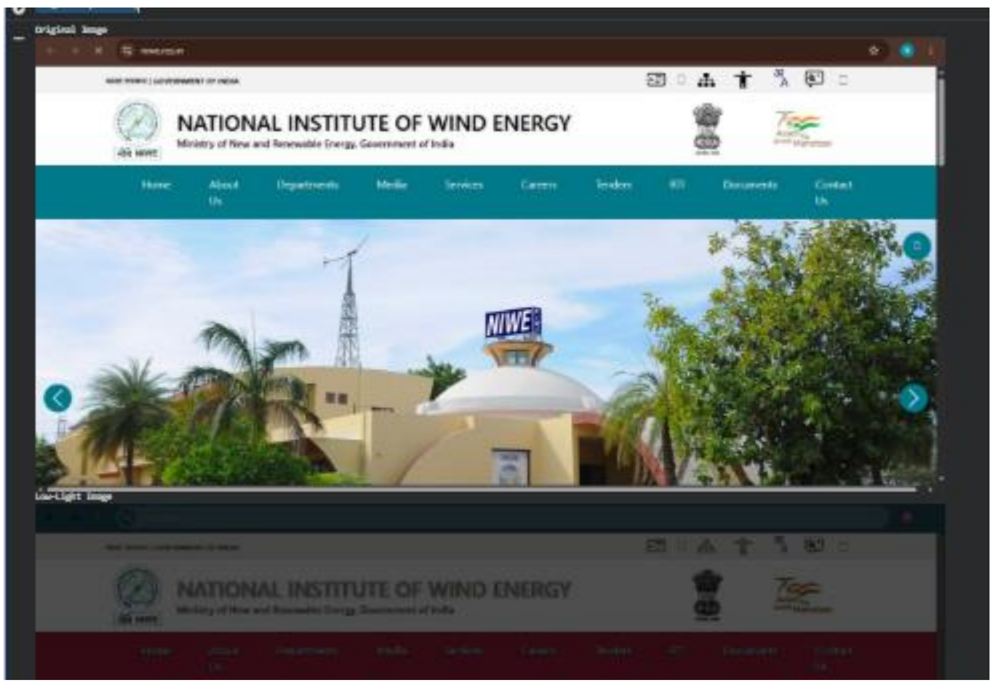
Q1.

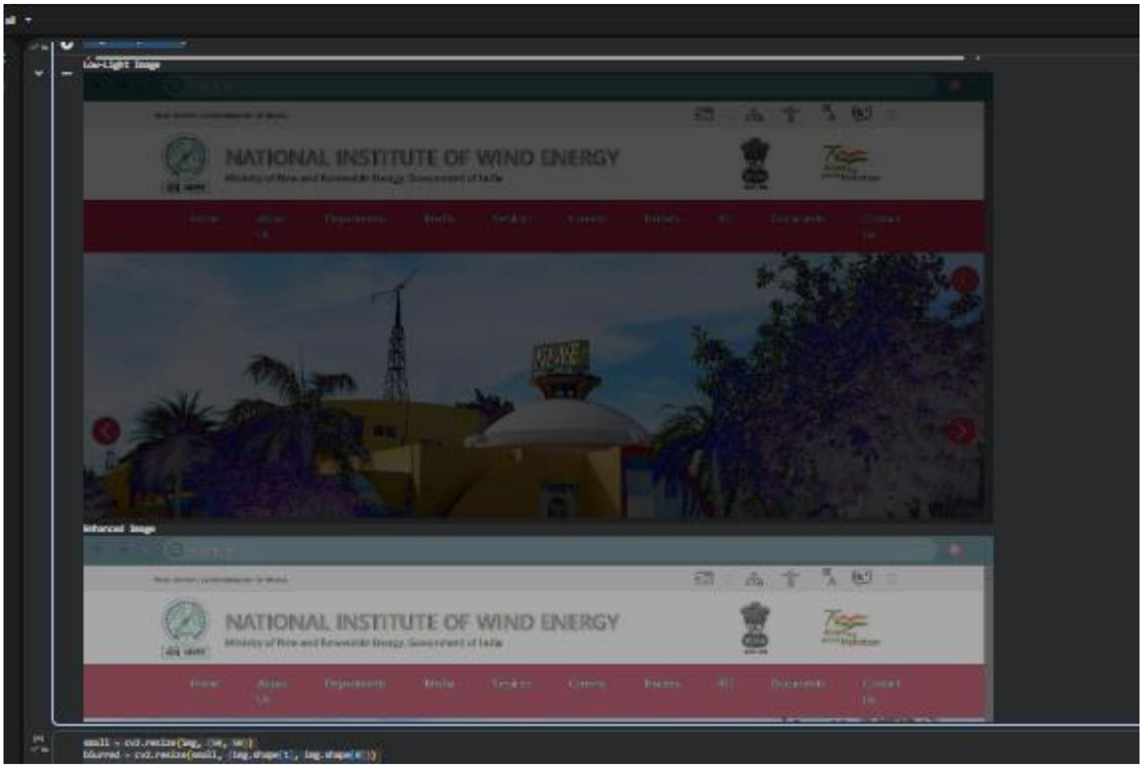
```
small = cv2.resize(img, (50, 50))
blurred = cv2.resize(small, (img.shape[1], img.shape[0]))

print("Original")
cv2_imshow(img)

print("Blurred")
cv2_imshow(blurred)
```

Q2.

```
import cv2
from google.colab.patches import cv2_imshow

# Read uploaded image
img = cv2.imread(filename)    # filename is obtained from the upload cell
```

```
# Simulate a low-light image by reducing brightness
low_light = cv2.convertScaleAbs(img, alpha=0.5, beta=-50)

# Enhance the low-light image
enhanced = cv2.convertScaleAbs(low_light, alpha=1.5, beta=50)

print("Original Image")
cv2_imshow(img)

print("Low-Light Image")
cv2_imshow(low_light)

print("Enhanced Image")
cv2_imshow(enhanced)
```

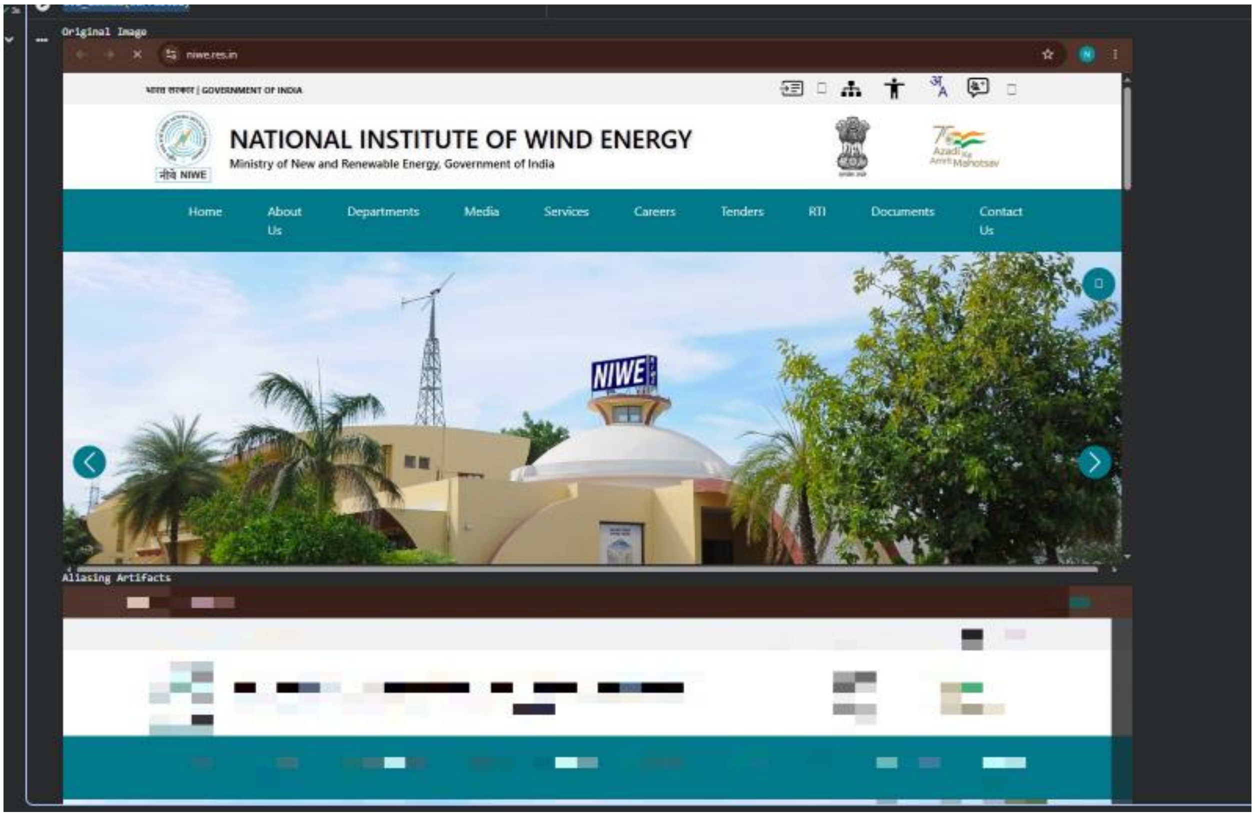




Q3.

```python
import cv2
from google.colab.patches import cv2_imshow

# Read uploaded image
img = cv2.imread(filename)

# Downsample the image (poor sampling)
small = cv2.resize(img, (50, 50))

# Upsample using nearest-neighbor interpolation (creates aliasing/blocky
artifacts)
aliasing = cv2.resize(small, (img.shape[1], img.shape[0]),
                      interpolation=cv2.INTER_NEAREST)

# Corrective approach: Use smoother interpolation
corrected = cv2.resize(small, (img.shape[1], img.shape[0]),
                       interpolation=cv2.INTER_LINEAR)

print("Original Image")
cv2_imshow(img)

print("Aliasing Artifacts")
cv2_imshow(aliasing)
```

```
print("Corrected Image")
cv2_imshow(corrected)
```



Q4.

```
import cv2
from google.colab.patches import cv2_imshow

# Read uploaded image
img = cv2.imread(filename)

# --------------------------------
# Low Pixel Resolution
# --------------------------------
small = cv2.resize(img, (80, 80))
low_pixel = cv2.resize(small, (img.shape[1], img.shape[0]))

# --------------------------------
# Low Intensity Resolution
# --------------------------------
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
# Reduce intensity levels (256 -> 4 levels)
low_intensity = (gray // 64) * 64

print("Original Image")
cv2_imshow(img)

print("Low Pixel Resolution")
cv2_imshow(low_pixel)

print("Low Intensity Resolution")
cv2_imshow(low_intensity)
```

Q5.

```
import cv2
from google.colab.patches import cv2_imshow

# Read uploaded image
img = cv2.imread(filename)

# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Load face detector
face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + "haarcascade_frontalface_default.xml"
)

# Detect faces
faces = face_cascade.detectMultiScale(gray, 1.3, 5)

# Draw rectangles around detected faces
for (x, y, w, h) in faces:
    cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 2)

print("Detected Faces")
cv2_imshow(img)
```

```
print("Number of faces detected:", len(faces))
```